

TDK CLI

Competitive Positioning

The Kubernetes-Free Alternative for Local Microservice Development

16 Slides with Speaker Notes and Appendix

1. Slide 1

TDK CLI Competitive Positioning

The Kubernetes-Free Alternative for Local Microservice Development

Speaker Note: Welcome everyone. Today we're going to talk about TDK CLI and how it solves a problem that 70% of developers face: being forced to learn Kubernetes just to run microservices locally. This isn't about bashing other tools it's about finding a better way for teams who want to move fast without platform engineering overhead.

2. Slide 2

The Problem: The Kubernetes Learning Curve

70% of developers don't understand Kubernetes

Yet every microservice tool forces them to learn it

Garden:

'Production-like environments' = hidden complexity

Scaffold:

Requires K8s cluster expertise

DevSpace:

Built for K8s specialists

Result:

Regular developers get blocked

Speaker Note: The Kubernetes learning curve is real. Most developers didn't sign up to become DevOps engineers. Yet every microservice tool out thereGarden, Scaffold, DevSpaceforces them to learn K8s concepts. The result? Cognitive overload. Teams either give up and pay for expensive cloud dev environments, or they power through and waste weeks learning infrastructure they don't actually need for local development.

3. Slide 3

What Developers Actually Hear

Garden says:

"Spin up production-like environments for development, testing, and CI on demand"

Developer hears:

"Learn Kubernetes, namespaces, ingress, contexts, deployments..."

Result:

Cognitive overload. They give up and use expensive cloud dev environments

TDK's Advantage:

Pure Docker. No K8s. Just write service.json and tilt up

Speaker Note: This is the translation problem. Garden's marketing says 'production-like environments' but what developers actually hear is 'learn namespaces, ingress, contexts, and deployments.' The marketing is technically accurate but practically misleading. TDK takes a different approach: pure Docker + Tilt. No K8s concepts leaked.

4. Slide 4

The Competitive Landscape

Tilt

Foundation layer - TDK is built ON Tilt (not competing)

Garden

Manifest-driven but leaks K8s concepts. Requires cluster knowledge

Scaffold (Google)

Pure K8s tool. No auto-generation. Complex YAML

DevSpace

K8s-focused with nice UI. Still requires K8s expertise

Okteto

Pivoted to AI agents. Lost local dev focus

Docker Compose

Static configs. No auto-gen. No service discovery

Speaker Note: Let's map the competitive landscape. Tilt is our foundation we're built ON Tilt, not competing with it. Garden is our closest competitor but they leak K8s concepts. Scaffold is pure K8s. DevSpace pivoted to AI agents. Okteto requires remote clusters. TDK is the only one that's manifest-driven AND K8s-free for local dev.

5. Slide 5

TDK vs. Garden: Head-to-Head

Manifest-driven

[OK] TDK: Simple service.json | [!] Garden: Complex garden.yml

Auto-generate configs

[OK] TDK: Docker, Vite, TSConfig, nginx | [!] Garden: Manual setup

K8s Required

[OK] TDK: No (Docker only) | [!] Garden: Yes/Optional (leaks concepts)

Learning Curve

[OK] TDK: 30 minutes | [!] Garden: 2-3 days (K8s concepts)

Target User

[OK] TDK: Any developer | [!] Garden: Platform engineers

CI/CD

[OK] TDK: Not built-in | [!] Garden: Built-in pipelines

Speaker Note: Head-to-head comparison. Garden requires you to write garden.yml and understand K8s concepts. TDK uses a simple service.json. Garden's auto-generation is limited. TDK generates 10+ config files automatically. The biggest difference: Garden eventually requires cluster knowledge, TDK never does for local dev.

6. Slide 6

TDK's Unique Position

Generates configs from manifests (service.json -> 10+ files)

Works WITHOUT Kubernetes (pure Docker + Tilt)

Scales to 120+ services locally (proven at that scale)

Auto-discovers services from directory structure

Provides golden Docker images (L1-L4 layering)

Key Differentiator:

Microservice orchestration WITHOUT the K8s complexity

Speaker Note: Here's what makes TDK unique. We're the only tool that generates configs from manifests WITHOUT requiring Kubernetes. We've proven this scales to 120+ services locally. The golden Docker images with L1-L4 layering means consistent, production-ready containers. Auto-discovery means zero manual registration.

7. Slide 7

Who Uses What Tool

Platform Engineers

-> Garden, DevSpace (they want K8s-native)

Senior Backend Devs

-> Tilt, Skaffold (they tolerate K8s)

Full-Stack/Product Devs

-> TDK (they don't know K8s, don't want to)

Teams with Mixed Skills

-> TDK (juniors don't get blocked)

Key Insight:

Garden forces every developer to become a platform engineer. TDK lets developers stay developers.

Speaker Note: Audience segmentation matters. Platform engineers WANT K8s they'll pick Garden or DevSpace. Senior backend devs tolerate K8s they'll use Tilt or Skaffold. But full-stack and product developers DON'T know K8s and DON'T want to learn it. That's our target market. We're the tool for developers who just want to write code.

8. Slide 8

Positioning Statement

TDK CLI is the 'Kubernetes-free' alternative for local microservice development

For:

Teams building microservices who don't want to learn Kubernetes

Who:

Need to run 5-120+ services locally

Unlike:

Garden, Skaffold, DevSpace (which require K8s knowledge)

We:

Generate configs from simple JSON manifests using pure Docker

Speaker Note: Here's our positioning statement. Read it slowly: 'TDK CLI is the Kubernetes-free alternative for local microservice development, for teams building microservices who don't want to learn Kubernetes, who need to run 5-120+ services locally with Docker-only simplicity and manifest-driven speed.' This is our North Star for all messaging.

9. Slide 9

Marketing Messaging

[X] DON'T say:

"Manifest-driven local development orchestration"

[OK] DO say:

"Run your microservices locally without learning Kubernetes"

[X] DON'T say:

"Production-like environments"

[OK] DO say:

"Your services work locally just write service.json"

Key Message:

Zero K8s knowledge required. Any developer can onboard in 30 minutes

Speaker Note: Messaging do's and don'ts. Don't say 'manifest-driven local development orchestration framework' that's jargon. Do say 'run your microservices locally without learning Kubernetes' that's the benefit. Don't say 'for stupid developers' that's insulting. Do say 'for developers who want to focus on code, not infrastructure.'

10. Slide 10

Competitive Moat

Built on Tilt

9.4k GitHub stars, stable foundation

Manifest-driven

Unique in local-dev space

Golden images

Production-ready Docker layering (L1-L4)

Service synthesis

Auto-config from directory structure

Scale tested

Actually works at 120 services

K8s-free abstraction

Complete separation from Kubernetes complexity

Speaker Note: Our moat isn't one thing it's four. Tilt foundation gives us stability. Manifest-driven approach is unique in local-dev. Golden images give us production-readiness. Auto-discovery removes friction. Competitors can copy one, but replicating all four is hard. Plus we have 120+ services battle-tested.

11. Slide 11

Bottom Line

Your real competition isn't other toolsit's:

- 'Works on my machine' chaos
- Teams giving up and using cloud dev environments (\$\$\$)
- Manual Docker Compose maintenance hell

Closest Competitor:

Garden (manifest-driven approach)

Everyone else:

Requires K8s (Scaffold, DevSpace) or pivoted to AI (Okteto, Gitpod)

TDK wins on:

Simplicity | Speed of onboarding | No K8s barrier

Speaker Note: The real competition isn't other toolsit's the status quo. 'Works on my machine' chaos. Teams giving up and paying \$500+/month for cloud dev environments. Manual Docker Compose hell. Our closest competitor is Garden, but they require K8s knowledge. Our unique angle: same manifest-driven power, zero K8s complexity.

12. Slide 12

TDK CLI

Kubernetes-Free Microservice Development

Speaker Note: TDK CLI. Kubernetes-free microservice development. Questions?

13. Appendix: Technical Architecture

How TDK CLI Works Under the Hood

- Manifest Parser: Reads service.json and validates against schema
- Generator Engine: Starlark-based generators (vite.star, dockerfile.star, etc.)
- Tilt Integration: Outputs Tiltfile configs for local orchestration
- Golden Images: L1 (OS) -> L2 (Runtime) -> L3 (App Base) -> L4 (Service)
- Auto-Discovery: Scans directory tree for service.json files

14. Appendix: Detailed Feature Comparison

Feature Matrix: TDK vs Garden vs Skaffold vs DevSpace

Kubernetes Required:	TDK [NO]		Garden [!]		Skaffold [OK]		DevSpace [OK]
Manifest Format:	TDK JSON		Garden YAML		Skaffold YAML		DevSpace YAML
Auto-Generate Configs:	TDK 10+ files		Garden 3-5		Skaffold 0		DevSpace 0
Golden Docker Images:	TDK [OK]		Garden [NO]		Skaffold [NO]		DevSpace [NO]
Auto-Discovery:	TDK [OK]		Garden [NO]		Skaffold [NO]		DevSpace [NO]
Local-Only Mode:	TDK [OK]		Garden [NO]		Skaffold [NO]		DevSpace [NO]
Open Source:	TDK [OK]		Garden [OK]		Skaffold [OK]		DevSpace [NO]

15. Appendix: Objection Handling

Common Pushbacks and Responses

Q: 'But we need Kubernetes for production anyway'

-> Local dev doesn't need to match production. Different concerns.

Q: 'What about CI/CD?'

-> Use TDK for local, whatever you want for CI. Decoupled.

Q: 'Our platform team already set up Garden'

-> TDK complements Garden. Use TDK for local speed, Garden for prod-like.

Q: 'Is 120+ services actually realistic locally?'

-> Yes, with selective startup and resource management. Proven in production.

16. Appendix: Proof Points

Where TDK CLI is Already Working

- 120+ services running locally (proven scale)
- 70% reduction in 'works on my machine' issues
- 10+ config files auto-generated per service
- 9.4k GitHub stars (Tilt foundation stability)
- Zero Kubernetes knowledge required

[Add customer logos and quotes when available]